

WIN11 OPTIMIZER

IMPLEMENTATION STRATEGY & IDEAS

Document 2 of 3 | Build Guide | v1.0

ASPECT	DESCRIPTION
Tool Type	TUI Application — separate terminal, ASCII-rendered interface
Inspired By	Chris Titus Tech WinUtil — but layered + reversible + safer
Language	PowerShell (backend) + Python or Batch (TUI front-end) + ANSI colors
Distribution	Single .ps1 or .exe — no external dependencies
Key Feature	Dry-run mode, live status, rollback engine, state snapshot

This document outlines the complete implementation plan for the Win11 Optimizer tool. It covers the TUI design, scripting strategy, module architecture, rollback system, and all PowerShell command patterns per layer. This is the 'how to build it' guide.

02 | MODULE ARCHITECTURE

The tool is organized as a modular PowerShell project. Each module is independent and can be sourced individually. The main script orchestrates all modules.

DIRECTORY STRUCTURE

```
C:\WinOptimizer\  
|-- WinOptimizer.ps1      # Main entry point + TUI engine  
|-- modules\  
|   |-- Snapshot.psm1     # Export system baseline state  
|   |-- Layer_Minimal.psm1 # All Minimal layer tweaks  
|   |-- Layer_Moderate.psm1 # All Moderate layer tweaks  
|   |-- Layer_Ultimate.psm1 # All Ultimate layer tweaks  
|   |-- Layer_GodMode.psm1 # All God Mode tweaks  
|   |-- Rollback.psm1     # Rollback engine + inverse commands  
|   |-- Verify.psm1       # Post-tweak verification checks  
|   |-- Status.psm1       # Live system status reporter  
|   |-- Logger.psm1       # Append-only change logger  
|-- snapshots\  
|-- logs\  
|-- config\  
|   |-- applied_layers.json # Which layers have been applied  
|   |-- user_prefs.json    # Skip list, custom settings  
|-- rollback\  
|   |-- rollback_minimal.psm1 # Auto-generated rollback per layer  
|   |-- rollback_moderate.psm1  
|   |-- rollback_ultimate.psm1  
|   |-- rollback_godmode.psm1
```

CORE MODULE RESPONSIBILITIES

MODULE	KEY FUNCTIONS	OUTPUT
WinOptimizer.ps1	Start-TUI, Show-Menu, Run-Layer	Terminal interface
Snapshot.psm1	Get-ServiceBaseline, Get-AppxBaseline	CSV/REG files
Layer_Minimal.psm1	Invoke-MinimalTweaks, Undo-Minimal	Applied changes
Layer_Moderate.psm1	Invoke-ModerateTweaks, Undo-Moderate	Applied changes
Layer_Ultimate.psm1	Invoke-UltimateTweaks, Undo-Ultimate	Applied changes
Layer_GodMode.psm1	Invoke-GodModeTweaks, Undo-GodMode	Applied changes
Rollback.psm1	Invoke-Rollback, Get-AppliedLayers	Reversed changes
Verify.psm1	Test-SystemIntegrity, Test-LayerState	Pass/Fail report
Status.psm1	Get-SystemStatus, Show-LivePanel	Live metrics
Logger.psm1	Write-Log, Get-SessionLog	Log entries

03 | TWEAK FUNCTION PATTERN

Every single tweak follows the same function pattern. This ensures consistency, dry-run support, logging, and rollback generation. Never write a raw command — always wrap it in this pattern.

STANDARD TWEAK FUNCTION TEMPLATE

```
function Invoke-Tweak-DisableDiagTrack {
    param([switch]$DryRun)

    $TweakName = "Disable DiagTrack Telemetry Service"
    $RollbackCmd = 'sc config DiagTrack start=auto; sc start DiagTrack'

    Write-Log -Level INFO -Message "Applying: $TweakName"

    if ($DryRun) {
        Write-Host "[DRY RUN] Would execute: sc stop DiagTrack" -ForegroundColor Cyan
        Write-Host "[DRY RUN] Would execute: sc config DiagTrack start=disabled" -ForegroundColor Cyan
        return
    }

    try {
        & sc stop DiagTrack 2>$null
        & sc config DiagTrack start=disabled 2>$null
        Write-Log -Level SUCCESS -Message "$TweakName - Applied OK"
        Register-Rollback -Layer "Minimal" -TweakName $TweakName -Command $RollbackCmd
        Write-Host " [OK] $TweakName" -ForegroundColor Green
    }
    catch {
        Write-Log -Level ERROR -Message "$TweakName - FAILED: $_"
        Write-Host " [FAIL] $TweakName - Check logs" -ForegroundColor Red
        Invoke-Rollback-Single -TweakName $TweakName
    }
}
```

The **Register-Rollback** function appends the inverse command to the layer's rollback script file. This means the rollback file is auto-built as you apply tweaks — you never have to write rollback code manually.

LOGGER FUNCTION

```
function Write-Log {
    param([string]$Level, [string]$Message)
    $timestamp = Get-Date -Format "yyyy-MM-dd HH:mm:ss"
    $entry = "$timestamp [$Level] $Message"
    Add-Content -Path "C:\WinOptimizer\logs\session.log" -Value $entry
    # Color output
    $color = @{ "INFO"="Cyan"; "SUCCESS"="Green"; "ERROR"="Red"; "WARN"="Yellow" }[$Level]
    Write-Host $entry -ForegroundColor $color
}
```

04 | LAYER IMPLEMENTATION — ALL POWERSHELL COMMANDS

Complete PowerShell command reference for each layer. These are the exact commands that the Tweak Engine executes.

[LAYER 1 — MINIMAL]

```
# Disable DiagTrack telemetry service
sc stop DiagTrack sc config DiagTrack start=disabled

ROLLBACK: sc config DiagTrack start=auto && sc start DiagTrack

# Disable background apps for all UWP
reg add "HKCU\Software\Microsoft\Windows\CurrentVersion\BackgroundAccessApplications" /v
GlobalUserDisabled /t REG_DWORD /d 1 /f

ROLLBACK: reg delete "HKCU\...\BackgroundAccessApplications" /v GlobalUserDisabled /f

# Disable Delivery Optimization P2P
reg add "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\DeliveryOptimization\Config" /v
DODownloadMode /t REG_DWORD /d 0 /f

ROLLBACK: Remove the DODownloadMode registry value

# Disable Xbox Game Bar
reg add "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\GameDVR" /v AppCaptureEnabled /t REG_DWORD
/d 0 /f

ROLLBACK: Set AppCaptureEnabled = 1

# Disable Widgets via policy
reg add "HKLM\SOFTWARE\Policies\Microsoft\Dsh" /v AllowNewsAndInterests /t REG_DWORD /d 0 /f

ROLLBACK: reg delete "HKLM\SOFTWARE\Policies\Microsoft\Dsh" /v AllowNewsAndInterests /f

# Disable Windows Spotlight
reg add "HKCU\Software\Policies\Microsoft\Windows\CloudContent" /v DisableWindowsSpotlightFeatures
/t REG_DWORD /d 1 /f

ROLLBACK: Delete the DisableWindowsSpotlightFeatures value

# Disable tips and suggestions
reg add "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\ContentDeliveryManager" /v
SubscribedContent-338388Enabled /t REG_DWORD /d 0 /f reg add
"HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\ContentDeliveryManager" /v
SubscribedContent-310093Enabled /t REG_DWORD /d 0 /f

ROLLBACK: Set both values back to 1
```

[LAYER 2 — MODERATE]

```
# Set telemetry to Level 0 via registry
New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\DataCollection" -Force Set-ItemProperty
-Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\DataCollection" -Name "AllowTelemetry" -Type DWord
-Value 0 -Force

ROLLBACK: Remove-Item "HKLM:\SOFTWARE\Policies\Microsoft\Windows\DataCollection" -Force

# Disable Advertising ID
Set-ItemProperty -Path "HKCU:\SOFTWARE\Microsoft\Windows\CurrentVersion\AdvertisingInfo" -Name
"Enabled" -Type DWord -Value 0 -Force

ROLLBACK: Set Enabled = 1

# Disable Xbox services
$svc = @("XblAuthManager","XblGameSave","XboxGipSvc","XboxNetApiSvc") foreach ($s in $svc) { if
(Get-Service -Name $s -EA SilentlyContinue) { Stop-Service $s -Force -EA SilentlyContinue;
Set-Service $s -StartupType Disabled } }

ROLLBACK: foreach ($s in $svc) { Set-Service $s -StartupType Manual }

# Disable MapsBroker, Fax, RetailDemo, WMPNetworkSvc
$svc2 = @("MapsBroker","Fax","RetailDemo","WMPNetworkSvc") foreach ($s in $svc2) { Stop-Service $s
-Force -EA SilentlyContinue; Set-Service $s -StartupType Disabled -EA SilentlyContinue }

ROLLBACK: Set each service back to Manual/Automatic

# Disable CEIP and Application Experience scheduled tasks
Disable-ScheduledTask -TaskPath "\Microsoft\Windows\Customer Experience Improvement Program\"
-TaskName "Consolidator" -EA SilentlyContinue Disable-ScheduledTask -TaskPath
"\Microsoft\Windows\Application Experience\" -TaskName "ProgramDataUpdater" -EA SilentlyContinue

ROLLBACK: Enable-ScheduledTask for each task path/name

# Remove Xbox + 3D Viewer + Solitaire UWP packages
Get-AppxPackage *Xbox* | Remove-AppxPackage Get-AppxPackage "Microsoft.Microsoft3DViewer" |
Remove-AppxPackage Get-AppxPackage "Microsoft.SolitaireCollection" | Remove-AppxPackage

ROLLBACK: Reinstall via Microsoft Store or Add-AppxPackage

# Disable SysMain on SSD
Stop-Service -Name "SysMain" -Force Set-Service -Name "SysMain" -StartupType Disabled

ROLLBACK: Set-Service SysMain -StartupType Automatic; Start-Service SysMain

# Add Defender exclusions for dev/data folders
Add-MpPreference -ExclusionPath "D:\datasets","D:\VMs","C:\Users\%USERNAME%\Projects\node_modules"

ROLLBACK: Remove-MpPreference -ExclusionPath
"D:\datasets","D:\VMs","C:\Users\%USERNAME%\Projects\node_modules"

# Uninstall OneDrive
$ods = "$env:SystemRoot\SysWOW64\OneDriveSetup.exe" if (Test-Path $ods) { & $ods /uninstall }

ROLLBACK: winget install Microsoft.OneDrive
```

[LAYER 3 — ULTIMATE]

```
# Block telemetry endpoint via WFP firewall
New-NetFirewallRule -DisplayName "Block MS Telemetry DiagTrack" -Direction Outbound -RemoteAddress "13.107.6.157" -Action Block -Profile Any
New-NetFirewallRule -DisplayName "Block MS Telemetry Watson" -Direction Outbound -RemoteAddress "65.52.100.9" -Action Block -Profile Any

ROLLBACK: Remove-NetFirewallRule -DisplayName "Block MS Telemetry DiagTrack"
Remove-NetFirewallRule -DisplayName "Block MS Telemetry Watson"

# Un-provision AppX for ALL users
$bl = @("Microsoft3DBuilder","Microsoft.MixedReality.Portal","Microsoft.XboxGamingOverlay","Microsoft.Xbox.TCUI")
Get-AppxProvisionedPackage -Online | Where-Object { $bl -contains $_.DisplayName } |
ForEach-Object { Remove-AppxProvisionedPackage -Online -PackageName $_.PackageName }

ROLLBACK: Re-provision via Store or in-place upgrade

# Disable hibernation
powercfg -h off

ROLLBACK: powercfg -h on

# Disable driver updates via Windows Update (GPO)
New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate" -Force Set-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate" -Name "ExcludeWUDriversInQualityUpdate" -Type DWord -Value 1

ROLLBACK: Delete ExcludeWUDriversInQualityUpdate registry value

# Disable additional CEIP tasks
Get-ScheduledTask | Where-Object { $_.TaskPath -match "Customer Experience" } |
Disable-ScheduledTask

ROLLBACK: Get-ScheduledTask | Where-Object { ... } | Enable-ScheduledTask

# Disable Windows Error Reporting
Stop-Service WerSvc -Force Set-Service WerSvc -StartupType Disabled

ROLLBACK: Set-Service WerSvc -StartupType Manual; Start-Service WerSvc
```

[LAYER 4 — GOD MODE]

```
# Disable ContentDeliveryManager consumer features
reg add "HKLM\SOFTWARE\Policies\Microsoft\Windows\CloudContent" /v DisableConsumerFeatures /t REG_DWORD /d 1 /f

ROLLBACK: reg delete "HKLM\SOFTWARE\Policies\Microsoft\Windows\CloudContent" /v DisableConsumerFeatures /f

# Disable Windows Copilot
reg add "HKCU\Software\Policies\Microsoft\Windows\WindowsCopilot" /v TurnOffWindowsCopilot /t REG_DWORD /d 1 /f

ROLLBACK: Delete TurnOffWindowsCopilot registry value

# Disable WSearch indexer completely
sc stop WSearch sc config WSearch start=disabled

ROLLBACK: sc config WSearch start=delayed-auto && sc start WSearch

# Disable OneSyncSvc (account settings sync)
Stop-Service OneSyncSvc -Force Set-Service OneSyncSvc -StartupType Disabled

ROLLBACK: Set-Service OneSyncSvc -StartupType Manual; Start-Service OneSyncSvc

# Disable Print Spooler
sc stop Spooler sc config Spooler start=disabled

ROLLBACK: sc config Spooler start=auto && sc start Spooler

# Disable background app infrastructure globally
reg add "HKCU\Software\Microsoft\Windows\CurrentVersion\BackgroundAccessApplications" /v GlobalUserDisabled /t REG_DWORD /d 1 /f

ROLLBACK: Delete GlobalUserDisabled value

# Remove full Xbox ecosystem
Get-AppxPackage *Xbox* | Remove-AppxPackage @("XblAuthManager","XblGameSave","XboxNetApiSvc") |
ForEach-Object { Set-Service $_ -StartupType Disabled -EA SilentlyContinue }

ROLLBACK: Reinstall Xbox from Store; re-enable services

# Disable SysMain completely
```

```
sc stop SysMain sc config SysMain start=disabled
```

```
ROLLBACK: sc config SysMain start=auto && sc start SysMain
```


05 | ROLLBACK ENGINE DESIGN

The rollback engine is the most important part of the tool. It must be reliable, idempotent, and testable. Every rollback command is pre-validated before being written to the rollback script.

ROLLBACK ARCHITECTURE

COMPONENT	FUNCTION	NOTES
Rollback Registry	JSON file mapping tweak → inverse cmd	Auto-updated per tweak
Rollback Scripts	Per-layer .ps1 files (auto-generated)	Run directly if TUI fails
System Restore	Checkpoint created before each layer	GUI: rstrui.exe
Disk Image	Full backup before Ultimate/GodMode	External tool required
Manual Rollback	User can run inverse commands manually	Documented in this guide

ROLLBACK FUNCTION EXAMPLE

```
function Invoke-Rollback-Layer {
    param([string]$Layer) # "Minimal", "Moderate", "Ultimate", "GodMode"

    $scriptPath = "C:\WinOptimizer\rollback\rollback_$(($Layer.ToLower())).ps1"

    if (-not (Test-Path $scriptPath)) {
        Write-Host "[WARN] No rollback script found for $Layer" -ForegroundColor Yellow
        return
    }

    Write-Host "Rolling back $Layer layer..." -ForegroundColor Cyan
    $confirm = Read-Host "This will reverse all $Layer tweaks. Type YES to confirm"
    if ($confirm -ne "YES") { Write-Host "Cancelled."; return }

    # Create restore point first
    Checkpoint-Computer -Description "pre-rollback-$Layer" -RestorePointType MODIFY_SETTINGS

    # Execute rollback script
    & $scriptPath

    # Update applied_layers.json
    $config = Get-Content "C:\WinOptimizer\config\applied_layers.json" | ConvertFrom-Json
    $config.applied = $config.applied | Where-Object { $_ -ne $Layer }
    $config | ConvertTo-Json | Set-Content "C:\WinOptimizer\config\applied_layers.json"

    Write-Host "[DONE] $Layer rollback complete. Reboot recommended." -ForegroundColor Green
}
```

EMERGENCY ROLLBACK SEQUENCE (if TUI crashes)

STEP	ACTION
Step 1	Boot into Safe Mode (Shift+Restart → Troubleshoot → Startup Settings)
Step 2	Open PowerShell as Administrator
Step 3	Run: Set-ExecutionPolicy Bypass -Scope Process

Step 4	Navigate: cd C:\WinOptimizer\rollback\
Step 5	Run the most recent layer's rollback: .\rollback_godmode.ps1
Step 6	Reboot normally — system should recover
Step 7	If still broken — use System Restore: rstrui.exe
Step 8	If restore fails — restore from disk image (Macrium / Clonezilla)
Step 9	Nuclear option: Windows in-place repair from install media

06 | LIVE STATUS PANEL DESIGN

The status panel shows real-time system health metrics. It runs as a background refresh loop and updates every 3 seconds in the TUI.

```
+-----+
|  SYSTEM STATUS                                WIN11 OPTIMIZER  |
+-----+
|  CPU Usage   : [#####-] 88%      Idle: 12%                |
|  RAM Usage   : 2.1 GB / 8 GB      Free: 5.9 GB             |
|  Processes   : 87 running                                     |
|  Background  : 12 services active                             |
|  Telemetry   : [BLOCKED]                                       |
|  Defender    : [ACTIVE]                                         |
|  Win Update  : [ACTIVE]                                         |
+-----+
|  APPLIED LAYERS                                     |
|  [x] MINIMAL  [x] MODERATE  [ ] ULTIMATE  [ ] GOD          |
+-----+
|  LAST CHANGE  : Disabled SysMain 12:43:21                 |
|  SNAPSHOT     : 2024-01-15 12:40:00 [OK]                   |
|  RESTORE PT   : pre-moderate-debloat [EXISTS]              |
+-----+
```

STATUS PANEL POWERSHELL QUERIES

METRIC	QUERY
CPU Usage	\$cpu = (Get-WmiObject Win32_Processor).LoadPercentage
RAM Usage	\$ram = Get-CimInstance Win32_OperatingSystem \$used = [math]::Round((\$ram.TotalVisibleMemorySize - \$ram.FreePhysicalMemory) / 1MB, 1)
Process Count	\$procs = (Get-Process).Count
Running Services	\$svcs = (Get-Service Where-Object {\$_.Status -eq 'Running'}).Count
Defender Status	\$def = (Get-MpComputerStatus).AntivirusEnabled
Applied Layers	\$layers = (Get-Content "C:\WinOptimizer\config\applied_layers.json" ConvertFrom-Json).applied

07 | IMPLEMENTATION TIPS & IDEAS

Beyond the core TUI and command engine, here are additional features and ideas to make this a truly production-grade tool:

IDEA	DESCRIPTION	PRIORITY
Per-Tweak Toggle	Let user skip individual tweaks within a layer	HIGH
Profile Presets	Student / Developer / Gamer presets with optimal layers	HIGH
Auto Restore Point	Always create restore point before any layer	HIGH
Diff Viewer	Show before/after service list comparison	MED
Scheduled Tasks Viewer	ASCII table of all enabled MS scheduled tasks	MED

App Dependency Check	Warn if removing an app that another app depends on	MED
Export Report	Generate a markdown/HTML report of all changes	LOW
Update Checker	Check GitHub for newer optimizer version	LOW
Safe Mode Launcher	Guide user to apply God Mode tweaks from Safe Mode	MED
Replacement Suggestions	After disabling WSearch, suggest Everything Search	HIGH
Hardware Profile	Detect SSD vs HDD, RAM size, and adjust suggestions	HIGH
VM Detection	Detect if running in VM and skip hardware-specific tweaks	MED

This is Document 2 of 3. See Document 3 for the testing methodology, verification checklist, and post-tweak validation procedures.